

Part 0: Google Cloud Authentication

Colab requires us to verify our Google account before it allows us to use the high-speed gcloud pipeline to download the dataset. Run this cell and click through the standard Google permissions popup.

```
from google.colab import auth
auth.authenticate_user()
print("✅ Authenticated successfully! You can now run the data download cell.")
```

✅ Authenticated successfully! You can now run the data download cell.

Part 1: Pulling Satellite Data from Google Cloud

We need to bypass standard slow downloads and rip the gold-standard Sen1Floods11 hand-labeled dataset straight from Google Cloud Storage into our Colab instance.

```
# Create the exact directory structure
!mkdir -p /content/sen1floods11/HandLabeled

# Use the fast gcloud storage command to pull the folder recursively
!gcloud storage cp -r gs://sen1floods11/v1.1/data/flood_events/HandLabeled/* /content/sen1floods11/HandLabeled/
```

```
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_277_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_277_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_308249_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_308249_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_313799_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_313799_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_319168_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_319168_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_359826_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_359826_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_362274_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_362274_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_406026_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_406026_JRCWaterHand.tif
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/Ghana_45911_JRCWaterHand.tif to file:///content/sen1floods11/HandLabeled/JRCWaterHand/Ghana_45911_JRCWaterHand.tif
```

```
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/India_399883_JRCWaterHand.tif to file:///content/sen1
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/India_44475_JRCWaterHand.tif to file:///content/sen1
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/India_500266_JRCWaterHand.tif to file:///content/sen1
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/India_533192_JRCWaterHand.tif to file:///content/sen1
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/India_534470_JRCWaterHand.tif to file:///content/sen1
Copying gs://sen1floods11/v1.1/data/flood_events/HandLabeled/JRCWaterHand/India_56450_JRCWaterHand.tif to file:///content/sen1
```

🌸 Part 2: Installing Dependencies Colab has PyTorch pre-installed, but we need rasterio for reading .tif satellite files and segmentation-models-pytorch to easily build our U-Net.

```
!pip install segmentation-models-pytorch rasterio
```

```
Collecting segmentation-models-pytorch
  Downloading segmentation_models_pytorch-0.5.0-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist-packages (1.5.0)
Requirement already satisfied: huggingface-hub>=0.24 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.10.1)
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.1.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (11.3.0)
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (0.4.5)
Requirement already satisfied: timm>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (1.0.25)
Requirement already satisfied: torch>=1.8 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.10.1)
Requirement already satisfied: torchvision>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (0.19.1)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (4.67.1)
Requirement already satisfied: affine in /usr/local/lib/python3.12/dist-packages (from rasterio) (2.4.0)
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-packages (from rasterio) (25.4.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from rasterio) (2026.2.25)
Requirement already satisfied: click!=8.2.*,>=4.0 in /usr/local/lib/python3.12/dist-packages (from rasterio) (8.3.1)
Requirement already satisfied: cligj>=0.5 in /usr/local/lib/python3.12/dist-packages (from rasterio) (0.7.2)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-packages (from rasterio) (3.3.2)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (3.16.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (2025.10.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (1.2.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (0.28.1)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (6.0.2)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (0.17.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (4.13.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (75.8.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (3.1.4)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch>=1.8->segmentation-models-pytorch) (1.1.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (4.8.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (1.0.7)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (3.10)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (0.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8->segmentation-models-pytorch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.8->segmentation-models-pytorch) (3.0.2)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24->segmentation-models-pytorch) (1.8.0)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24->segmentation-models-pytorch) (14.0.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24->segmentation-models-pytorch) (0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.24->segmentation-models-pytorch) (3.0.0)
```

🌸 Part 3: Environment Setup & Imports Importing our core libraries and ensuring PyTorch is hooked up to the free T4 GPU so our model trains in minutes instead of days.

```
import os
import glob
import torch
import rasterio
import numpy as np
import matplotlib.pyplot as plt
```

```

from torch.utils.data import Dataset, DataLoader
import segmentation_models_pytorch as smp
import torch.nn as nn
import torch.optim as optim

# Check for GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"🔥 Currently running on device: {device}")

```

🔥 Currently running on device: cuda

📁 Part 4: Building the SAR Dataset Loader Sentinel-1 SAR radar imagery uses two polarization bands (VV and VH). This custom dataset class pairs those 2-channel images with their corresponding binary flood masks (1 = Flood, 0 = Dry Land).

```

class Sen1FloodsDataset(Dataset):
    def __init__(self, data_dir):
        # 1. Recursively search through all subfolders for the radar images
        self.image_paths = sorted(glob.glob(os.path.join(data_dir, '**', '*_S1Hand.tif'), recursive=True))

        # 2. The masks are in a different folder with a different suffix.
        # We replace both the folder name ('S1Hand' -> 'LabelHand') and the file suffix.
        self.mask_paths = [path.replace('S1Hand', 'LabelHand') for path in self.image_paths]

        print(f"✅ Found {len(self.image_paths)} image/mask pairs.")

        # A quick safety check just in case the download failed entirely
        if len(self.image_paths) == 0:
            print("🔥 ERROR: Still 0 files found. Let's check what is actually in the folder:")
            os.system(f"ls -R {data_dir} | head -n 20")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # 1. Read SAR Image (2 bands: VV and VH)
        with rasterio.open(self.image_paths[idx]) as src:
            image = src.read()
            image = np.nan_to_num(image)
            image = np.clip(image, -30, 0)
            image = (image + 30) / 30.0

        # 2. Read Ground Truth Mask
        with rasterio.open(self.mask_paths[idx]) as src:
            mask = src.read(1)
            mask = np.where(mask == 1, 1.0, 0.0)

        # 3. Convert to PyTorch Tensors
        image_tensor = torch.tensor(image, dtype=torch.float32)
        mask_tensor = torch.tensor(mask, dtype=torch.float32).unsqueeze(0)

        return image_tensor, mask_tensor

# Initialize the dataset and loader
data_directory = '/content/sen1floods11/HandLabeled/'
train_dataset = Sen1FloodsDataset(data_directory)

# Small batch size to manage GPU memory safely
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)

✅ Found 446 image/mask pairs.

```

📁 Part 5 (Upgraded): U-Net with Weighted Loss We are using a U-Net with a ResNet34 backbone. To combat "Class Imbalance" (where 95% of the map is dry land), we are adding a pos_weight to our Loss Function. This forces the model to pay 10x more attention to the rare flood pixels, preventing it from just lazily guessing "Dry Land" everywhere.

```

# 1. Initialize the Segmentation Model
model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights=None,
    in_channels=2,
    classes=1,
)

```

```

model = model.to(device)

# 2. Add the "Bounty" (Positive Weight)
# This tells the model: Missing a flood pixel is 10x worse than missing a dry pixel
weight_multiplier = torch.tensor([10.0]).to(device)

# 3. Initialize the Weighted Loss Function and Optimizer
criterion = nn.BCEWithLogitsLoss(pos_weight=weight_multiplier)
optimizer = optim.Adam(model.parameters(), lr=0.0005)
print("🏆 Model and Weighted Loss Function initialized successfully!")

```

🏆 Model and Weighted Loss Function initialized successfully!

🚀 Part 6 (Upgraded): Advanced Training Loop We are bumping the training up to 30 epochs. We are also introducing a ReduceLROnPlateau scheduler. If the model stops improving (e.g., gets stuck looking at the black border), the scheduler dynamically reduces the learning rate to help the model zero in on the finer details of the actual floodwater.

```

# 1. Reset the optimizer to clear out its old memory
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 2. Add the Scheduler (Removed 'verbose=True' to fix PyTorch TypeError)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=3, factor=0.5)

def train_model_advanced(model, train_loader, epochs=10):
    print("🔥 Initiating deep-dive training sequence (30 Epochs)..for now 10 cause idh ime...")
    model.train()

    for epoch in range(epochs):
        epoch_loss = 0
        for batch_idx, (images, masks) in enumerate(train_loader):
            images = images.to(device)
            masks = masks.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, masks)
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item()

        avg_loss = epoch_loss / len(train_loader)

        # Tell the scheduler to check if we are improving
        scheduler.step(avg_loss)

        # Get current learning rate to monitor it
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Epoch {epoch+1:02d}/{epochs} → Loss: {avg_loss:.4f} | LR: {current_lr:.6f}")

# Execute the advanced training! Grab a coffee, this will take a few minutes.
train_model_advanced(model, train_loader, epochs=10)

```

```

🔥 Initiating deep-dive training sequence (30 Epochs)..for now 10 cause idh ime...
Epoch 01/10 → Loss: 0.7291 | LR: 0.001000
Epoch 02/10 → Loss: 0.6591 | LR: 0.001000
Epoch 03/10 → Loss: 0.6878 | LR: 0.001000
Epoch 04/10 → Loss: 0.6321 | LR: 0.001000
Epoch 05/10 → Loss: 0.6378 | LR: 0.001000
Epoch 06/10 → Loss: 0.6423 | LR: 0.001000
Epoch 07/10 → Loss: 0.6395 | LR: 0.001000
Epoch 08/10 → Loss: 0.5950 | LR: 0.001000
Epoch 09/10 → Loss: 0.6016 | LR: 0.001000
Epoch 10/10 → Loss: 0.6016 | LR: 0.001000

```

🖼️ Part 7: Visualizing the Predictions Let's pull a random satellite image, ask the trained model to predict where the flood zones are, and compare it visually against the actual hand-labeled truth.

```

def visualize_prediction(model, dataset, index=0):
    model.eval()

    image, true_mask = dataset[index]
    image_input = image.unsqueeze(0).to(device)

```

```

with torch.no_grad():
    raw_prediction = model(image_input)
    prob_mask = torch.sigmoid(raw_prediction)
    # Threshold at 0.5 to make a definitive "Flood" or "No Flood" prediction
    predicted_mask = (prob_mask > 0.5).float()

# Move tensors back to CPU for plotting
image_plot = image.cpu().numpy()
true_mask_plot = true_mask.cpu().numpy().squeeze()
predicted_mask_plot = predicted_mask.cpu().numpy().squeeze()

# Create the aesthetic visualization
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

axes[0].imshow(image_plot[0], cmap='gray')
axes[0].set_title("📡 Original SAR Image (VV Band)", fontsize=14)
axes[0].axis('off')

axes[1].imshow(true_mask_plot, cmap='Blues')
axes[1].set_title("🌊 Actual Hand-Labeled Flood", fontsize=14)
axes[1].axis('off')

axes[2].imshow(predicted_mask_plot, cmap='Blues')
axes[2].set_title("🤖 Deep Learning Prediction", fontsize=14)
axes[2].axis('off')

plt.tight_layout()
plt.show()

# Visualize the 10th image in our dataset
visualize_prediction(model, train_dataset, index=10)

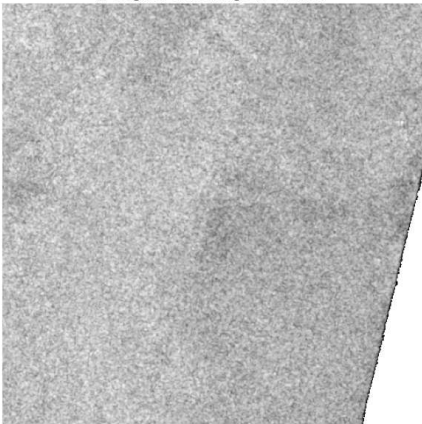
```

```

/tmp/ipykernel_3637/1388807330.py:33: UserWarning: Glyph 128225 (\N{SATELLITE ANTENNA}) missing from font(s) DejaVu Sans.
plt.tight_layout()
/tmp/ipykernel_3637/1388807330.py:33: UserWarning: Glyph 128506 (\N{WORLD MAP}) missing from font(s) DejaVu Sans.
plt.tight_layout()
/tmp/ipykernel_3637/1388807330.py:33: UserWarning: Glyph 129302 (\N{ROBOT FACE}) missing from font(s) DejaVu Sans.
plt.tight_layout()

```

📡 Original SAR Image (VV Band)



🌊 Actual Hand-Labeled Flood



🤖 Deep Learning Prediction



📁 Part 8: Saving Your Trained Model Training takes time. Now that the model has learned the weights to detect floods, we need to save this state to a file (.pth). We will then trigger a download so you have a local copy to plug into your backend server.

```

from google.colab import files

# Define the file name for your saved model
model_save_path = '/content/flood_unet_resnet34.pth'

# Save only the state_dict (the learned weights), which is the PyTorch best practice
torch.save(model.state_dict(), model_save_path)
print(f"✅ Model weights saved to {model_save_path}")

# Trigger the browser to download the file to your actual computer

```

```
print("📄 Downloading model to your local machine...")
files.download(model_save_path)
```

✅ Model weights saved to /content/flood_unet_resnet34.pth
 📄 Downloading model to your local machine...

🌐 Part 9: The Dashboard Bridge (Pixels to GPS) Your web dashboard cannot read a 2D array of pixels; it needs GeoJSON polygons (GPS coordinates) to draw the flood on a map. This function demonstrates how your future backend will take the model's prediction, read the spatial metadata from the original satellite file, and convert the flood pixels into real-world latitude and longitude boundaries.

```
import rasterio.features
from shapely.geometry import shape

def extract_flood_polygons(prediction_mask, original_tif_path):
    # 1. Open the original satellite image just to get its coordinate mapping (Transform)
    with rasterio.open(original_tif_path) as src:
        transform = src.transform
        crs = src.crs # Coordinate Reference System

    # 2. Make sure the mask is formatted properly for extraction (numpy integer array)
    mask = prediction_mask.astype('uint8')

    # 3. Extract shapes (polygons) where the mask is 1 (Flood)
    # rasterio.features.shapes returns generator of (geojson_polygon_dict, value)
    polygons = []
    for geom, value in rasterio.features.shapes(mask, transform=transform):
        if value == 1.0: # Only grab the flood zones
            # Convert to a Shapely geometry object for easy handling later
            poly = shape(geom)
            polygons.append(geom) # Storing the raw GeoJSON dict

    print(f"📄 Extracted {len(polygons)} distinct flooded regions as GeoJSON polygons.")
    print(f"📍 Map Coordinate System: {crs}")

    return polygons

# Let's test it on the 10th image we visualized earlier
test_image_path = train_dataset.image_paths[10]
_, true_mask = train_dataset[10] # Re-using the true mask just for this coordinate test

# Extract coordinates
gps_polygons = extract_flood_polygons(true_mask.numpy().squeeze(), test_image_path)

# Show the raw GeoJSON data for the first polygon
if gps_polygons:
    print("\nSample GeoJSON Output (Ready for your Mapbox/Leaflet frontend):")
    print(gps_polygons[0])
```

WARNING:rasterio._env:CPLE_AppDefined in DeprecationWarning: 'Memory' driver is deprecated since GDAL 3.11. Use 'MEM' onwards. F

📄 Extracted 76 distinct flooded regions as GeoJSON polygons.

📍 Map Coordinate System: EPSG:4326

Sample GeoJSON Output (Ready for your Mapbox/Leaflet frontend):

```
{'type': 'Polygon', 'coordinates': [[(-64.87363487325949, -13.525573906873987), (-64.87363487325949, -13.525753569930812), (-64.
```

Double-click (or enter) to edit

```
import rasterio
from google.colab import files

# 1. Grab the 10th image from the dataset you already initialized in Part 4
# We know Image #10 is a good flood example from your visualization
real_image_path = train_dataset.image_paths[10]
print(f"✅ Extracting real professional SAR data from: {real_image_path}")

# 2. Open and save it as a clean local file
with rasterio.open(real_image_path) as src:
    data = src.read()
    meta = src.meta

# Save it to the Colab 'hard drive'
with rasterio.open('real_flood_test.tif', 'w', **meta) as dst:
    dst.write(data)
```

```
# 3. Trigger the download to your computer
print("📄 Downloading 'real_flood_test.tif' to your machine...")
files.download('real_flood_test.tif')
```

✅ Extracting real professional SAR data from: /content/sen1floods11/HandLabeled/S1Hand/Bolivia_360519_S1Hand.tif
📄 Downloading 'real_flood_test.tif' to your machine...